

Apuntes de Clase: Semana 3 Jueves

Jose Miguel Gonzalez Barrantes
Carné: 2023087564
Escuela de Ingenieria en Computación
Instituto Tecnológico de Costa Rica
Profesor: Steven Andrey Pacheco Portugués
Curso: IC6200 Inteligencia Artificial
Fecha: Miércoles 11 de marzo de 2026

Resumen—Este documento es el resumen sobre lo visto en clase virtual del día 5 de marzo, en esta clase se repasó los conceptos de algebra lineal, después seguido de una explicación profunda sobre la programación vectorial con NumPy y la manipulación de datos con Pandas. Adicionalmente se aprendió sobre la implementación de K-NEarest Neighbors (KNN).

I. ÁLGEBRA LINEAL

I-A. Vectores

Un vector es un objeto matemático con dirección y magnitud:

$$\mathbf{x} = (x_1, x_2, \dots, x_n).$$

I-B. Magnitud (Norma Euclidiana)

$$\|\mathbf{x}\| = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}.$$

I-C. Vector entre dos puntos

Si $A = (a_1, \dots, a_n)$ y $B = (b_1, \dots, b_n)$, el vector de A a B es:

$$\mathbf{u} = B - A = (b_1 - a_1, \dots, b_n - a_n).$$

I-D. Distancias

Manhattan:

$$d_1(A, B) = \sum_{i=1}^n |a_i - b_i|.$$

Euclidiana (Pitágoras):

$$d_2(A, B) = \sqrt{\sum_{i=1}^n (a_i - b_i)^2}.$$

I-E. Norma (propiedades)

Una norma $\|\cdot\|$ cumple:

1. **Positividad:** $\|x\| \geq 0$ y $\|x\| = 0 \iff x = 0$.
2. **Homogeneidad:** $\|ax\| = |a| \|x\|$ para todo escalar a .
3. **Desigualdad triangular:** $\|x + y\| \leq \|x\| + \|y\|$.

I-F. Producto punto

Para $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$:

$$\mathbf{x} \cdot \mathbf{y} = \sum_{i=1}^n x_i y_i = \|\mathbf{x}\| \|\mathbf{y}\| \cos \theta.$$

I-G. Ángulo, ortogonalidad y ortonormalidad

$$\cos \theta = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}.$$

Dos vectores son ortogonales si $\mathbf{x} \cdot \mathbf{y} = 0$. Un conjunto es ortonormal si sus vectores son mutuamente ortogonales y cada uno tiene norma 1.

II. PROGRAMACIÓN VECTORIAL Y MANIPULACIÓN DE DATOS

II-A. Introducción a NumPy

NumPy es la biblioteca base para computación numérica en Python. Permite tener:

- Arrays multidimensionales eficientes.
- Operaciones vectorizadas.
- Funciones de álgebra lineal, estadísticas y transformaciones.
- Integración otras herramientas útiles con Pandas, Matplotlib y SciPy.

II-B. Creación de arrays (ejemplos)

A continuación se muestran ejemplos comunes para crear arrays en NumPy: secuencias con `arange`, arrays constantes con `ones` y `zeros`, y matrices como la identidad. Estos fragmentos sirven para ilustrar la sintaxis básica y los tipos de estructuras que se usan en programación vectorial.

```
1 import numpy as np
2
3 # 1D: arange
4 array_1d = np.arange(10)          # [0 1 2 ...
5                                     9]
6 array_1d = np.arange(5,10)        # [5 6 7 8
7                                     9]
8 array_1d = np.arange(5,10,2)      # [5 7 9]
9
10 # Arrays constantes
11 ones = np.ones(10)
12 zeros = np.zeros(10)
13
14 # Matrices e identidades
15 I = np.identity(3)
16 A = np.ones((4,5))
17 B = np.ones((2,4,5)) # tensor 3D
```

Listing 1: Creación básica de arrays con NumPy

II-C. Operaciones básicas

Las operaciones básicas aplican la misma operación a cada par de elementos de arrays con la misma forma; son prácticamente la base de la programación vectorial y evitan bucles.

```
1 lst = [2,4,6]
2 lst2 = [1,3,5]
3 a = np.array(lst)
4 b = np.array(lst2)
5
6 suma = a + b          # [3 7 11]
7 resta = a - b         # [1 1 1]
8 mult = a * b          # [2 12 30]
9 div = a / b           # [2. 1.333... 1.2]
```

Listing 2: Operaciones básicas

II-D. Operaciones por dimensiones

Según lo visto en clase, muchas de las funciones de NumPy aceptan el parámetro `axis` para agregar a lo largo de filas o columnas; esto permite cálculos eficientes por dimensión.

```
1 matriz_2d = np.array([[1,2,3],
2                        [4,5,6],
3                        [7,8,9]])
4
5 total = np.sum(matriz_2d)          # 45
```

```
6 col_sum = np.sum(matriz_2d, axis=0) # [12 15
7                                     18]
8 row_sum = np.sum(matriz_2d, axis=1) # [6 15
9                                     24]
```

Listing 3: Suma por filas/columnas

II-E. Funciones estadísticas

NumPy nos incluye funciones para calcular estadísticas básicas (media, desviación estándar, máximos y mínimos) de manera eficiente sobre arrays.

```
1 array_random = np.random.rand(100)
2 media = np.mean(array_random)
3 std = np.std(array_random)
4 maxv = np.max(array_random)
5 minv = np.min(array_random)
```

Listing 4: Estadísticos básicos

III. INTRODUCCIÓN A PANDAS Y DATASET DE EJEMPLO

III-A. Generación de dataset sintético

La función `createDataset` genera un conjunto sintético de flores con dos características y tres clases, útil para practicar clasificación y visualización.

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 def createDataset(samples, seed=42, plot=
6     True, isTraining=True):
7     np.random.seed(seed)
8     clase_1 = np.random.normal(loc=2, scale
9         =1, size=(samples//3, 2))
10    clase_2 = np.random.normal(loc=7, scale
11        =1.5, size=(samples//3, 2))
12    clase_3 = np.random.normal(loc=12, scale
13        =1, size=(samples//3, 2))
14    etiquetas = np.concatenate((
15        np.full(samples//3, 0),
16        np.full(samples//3, 1),
17        np.full(samples//3, 2)
18    ))
19    dataset = np.vstack((clase_1, clase_2,
20        clase_3))
21    columnas = ['Longitud_sepalo', '
22        Longitud_petalo']
23    df = pd.DataFrame(dataset, columns=
24        columnas)
25    if isTraining:
26        df['Clase'] = etiquetas
27        df = df.sample(frac=1).reset_index(
28            drop=True)
29    if plot:
```

```

22 plt.scatter(clase_1[:,0], clase_1
    [:,1], label='Clase 1', alpha
    =0.7)
23 plt.scatter(clase_2[:,0], clase_2
    [:,1], label='Clase 2', alpha
    =0.7)
24 plt.scatter(clase_3[:,0], clase_3
    [:,1], label='Clase 3', alpha
    =0.7)
25 plt.legend(); plt.xlabel('Longitud
    s palo'); plt.ylabel('Longitud
    p talo'); plt.show()
26 return df
27
28 dataFrameTraining = createDataset(300)

```

Listing 5: Función para crear dataset sintético

III-B. Exploración básica con Pandas

Se puede usar `head()`, `tail()`, `count()` y `describe()` para inspeccionar rápidamente la estructura y estadísticas del `DataFrame`.

```

1 dataFrameTraining.head()
2 dataFrameTraining.tail()
3 dataFrameTraining.count()
4 dataFrameTraining.describe()

```

Listing 6: Comandos básicos de inspección en Pandas

IV. K-NEAREST NEIGHBORS (KNN)

IV-A. Idea base

En general consiste en tomar un individuo del conjunto de datos de producción y otorgarle una clasificación basado en los datos que sí conozco.

IV-B. Implementación

La clase KNN calcula distancias euclidianas entre cada punto de prueba y todos los puntos de entrenamiento, selecciona los k índices más cercanos y predice la etiqueta por mayoría.

```

1 from scipy.stats import mode
2 import numpy as np
3
4 class KNN():
5     def __init__(self, dataFrameTraining,
6         dataFrameTesting, number_neighbors
7         =5, classes=3):
8         self.dataFrameTraining =
9             dataFrameTraining
10        self.dataFrameTesting =
11            dataFrameTesting

```

```

8 self.number_neighbors =
    number_neighbors
9 self.classes = classes
10 self.neigh_ind = []
11
12 def euclidean_distance(self, a, b):
13     # a: (n_samples, n_features), b: (
14         n_features,) -> devuelve vector
15         de distancias
16     return np.sqrt(np.sum((a - b) ** 2,
17         axis=1))
18
19 def computeDistances(self):
20     point_dist = []
21     for _, data in self.dataFrameTesting
22         .iterrows():
23         dist = self.euclidean_distance(
24             self.dataFrameTraining[["
25                 Longitud_sepalo", "
26                 Longitud_petalo"]].
27                 values,
28             data[["Longitud_sepalo", "
29                 Longitud_petalo"]].
30                 values
31         )
32     point_dist.append(dist)
33     # point_dist es lista de vectores de
34     distancias (uno por cada test
35     sample)
36     self.point_dist = np.array(
37         point_dist)
38
39 def predict(self):
40     labels_pred = []
41     for dist_vector in self.point_dist:
42         neigh_idx = np.argsort(
43             dist_vector)[:self.
44                 number_neighbors]
45         neigh_labels = self.
46             dataFrameTraining.iloc[
47                 neigh_idx][['Clase']].values
48         pred_label = mode(neigh_labels).
49             mode[0]
50         labels_pred.append(pred_label)
51     return np.array(labels_pred)

```

Listing 7: Clase KNN

IV-C. Usos

Entre sus usos vistos en clase están: instanciar el clasificador, calcular distancias y obtener predicciones para añadirlas al `DataFrame` de prueba.

```

1 classifier = KNN(dataFrameTraining,
2     dataFrameTesting, number_neighbors=3)
3 classifier.computeDistances()
4 labels_pred = classifier.predict()
5 dataFrameTesting['ClasePredicha'] =
6     labels_pred

```

Listing 8: Uso de la clase KNN

Se pueden visualizar las clases con `matplotlib` para inspección cualitativa, además de calcular métricas (accuracy, matriz de confusión) para evaluación cuantitativa.

APÉNDICE: SALIDAS DE LOS EJEMPLOS DE CLASE

```
Array 1D: [0 1 2 3 4 5 6 7 8 9]
```

```
Array 1D: [5 6 7 8 9]
```

```
Array 1D: [5 7 9]
```

```
Array 1D: [1. 1. 1. 1. 1. 1. 1. 1. 1. 1.]
```

```
Array 1D: [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
```

```
Identity 3x3:
```

```
[[1. 0. 0.]
```

```
 [0. 1. 0.]
```

```
 [0. 0. 1.]]
```

```
Suma total matriz: 45
```

```
Suma por columnas: [12 15 18]
```

```
Suma por filas: [ 6 15 24]
```